

Parabolic Equation - Forward-Difference và Backward-Difference Solver

Tóm tắt từ bài giảng Computational Physics và code thực hành

2026-06-23

Mục lục

1. Parabolic equation là gì?	3
2. Rời rạc hóa miền tính toán	3
3. Sai phân đạo hàm bậc hai theo không gian	4
4. Forward-Difference method	4
4.1 Ý tưởng	4
4.2 Điều kiện ổn định	5
4.3 Độ chính xác	5
4.4 Solver Forward-Difference	5
5. Backward-Difference method	6
5.1 Ý tưởng	6
5.2 Dạng ma trận	7
5.3 Tính chất	7
5.4 Solver Backward-Difference dùng Gauss-Seidel	7
6. Hàm lưu file kết quả	8
7. Ví dụ 1: một thanh ban đầu 100 độ C, hai đầu giữ 0 độ C	8
8. Ví dụ 2: hai thanh tiếp xúc	9
9. Vẽ đồ thị kết quả	10
9.1 Vẽ nhiệt độ theo vị trí tại vài thời điểm	10
9.2 Vẽ heatmap $u(x, t)$	10
10. So sánh Forward-Difference và Backward-Difference	11
11. Những lưu ý quan trọng trong code	11
11.1 Thứ tự chỉ số mảng	11
11.2 Điều kiện đầu và điều kiện biên có thể đè nhau	11
11.3 Forward method cần kiểm tra eta	12
11.4 Backward method cần kiểm tra hội tụ	12
11.5 Ổn định không đồng nghĩa với chính xác	12
12. Các dạng toán có thể áp dụng	12
12.1 Truyền nhiệt trong thanh 1D	12
12.2 Hai thanh tiếp xúc	12
12.3 Khuếch tán chất	13
12.4 Khuếch tán xác suất	13
12.5 Bài toán 2D	13

13. Chiến lược chọn phương pháp	13
14. Công thức cần nhớ	14
15. Kết luận	14
Tài liệu nguồn	15

1. Parabolic equation là gì?

Parabolic equation là nhóm phương trình đạo hàm riêng mô tả quá trình khuếch tán theo thời gian. Ví dụ kinh điển nhất là **phương trình truyền nhiệt**.

Dạng 2D thường gặp:

$$\alpha^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = \frac{\partial u}{\partial t}.$$

Dạng 1D:

$$\alpha^2 \frac{\partial^2 u(x, t)}{\partial x^2} = \frac{\partial u(x, t)}{\partial t}, \quad 0 < x < l, \quad t > 0.$$

Trong bài toán truyền nhiệt:

$$\frac{K}{C\rho} \frac{\partial^2 u}{\partial x^2} = \frac{\partial u}{\partial t},$$

với:

$$\alpha^2 = \frac{K}{C\rho}.$$

Trong đó:

Ký hiệu	Ý nghĩa
$u(x, t)$	nhiệt độ tại vị trí x và thời gian t
K	độ dẫn nhiệt
C	nhiệt dung riêng
ρ	khối lượng riêng
α^2	hệ số khuếch tán nhiệt
l	chiều dài thanh
$t_{\max}$	thời gian mô phỏng

2. Rời rạc hóa miền tính toán

Ta chia không gian và thời gian thành các điểm lưới:

$$x_i = ih, \quad i = 0, 1, \dots, N_x - 1,$$

$$t_j = jk, \quad j = 0, 1, \dots, N_t - 1.$$

Bước lưới:

$$h = \frac{l}{N_x - 1}, \quad k = \frac{t_{\max}}{N_t - 1}.$$

Ký hiệu nghiệm số:

$$u_{i,j} \equiv u(x_i, t_j).$$

Trong code Python nên dùng:

```
u = np.zeros((N_x, N_time), dtype=float)
```

và hiểu rằng:

```
u[i, j] = u(x_i, t_j)
i: chỉ số không gian x
j: chỉ số thời gian t
```

Lưu ý: nếu mảng có shape (N_x, N_time) thì **chỉ số đầu là không gian, chỉ số thứ hai là thời gian**.

3. Sai phân đạo hàm bậc hai theo không gian

Cả Forward-Difference và Backward-Difference đều dùng sai phân trung tâm cho đạo hàm bậc hai theo x :

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}.$$

Công thức này có sai số theo không gian bậc hai, tức là $O(h^2)$.

4. Forward-Difference method

4.1 Ý tưởng

Forward-Difference dùng sai phân tiến theo thời gian:

$$\frac{\partial u}{\partial t} \approx \frac{u_{i,j+1} - u_{i,j}}{k}.$$

Thay vào phương trình truyền nhiệt:

$$\alpha^2 \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} = \frac{u_{i,j+1} - u_{i,j}}{k}.$$

Đặt:

$$\eta = \frac{\alpha^2 k}{h^2}.$$

Ta thu được:

$$u_{i,j+1} = u_{i,j} + \eta (u_{i+1,j} - 2u_{i,j} + u_{i-1,j}).$$

Hay:

$$u_{i,j+1} = (1 - 2\eta)u_{i,j} + \eta u_{i+1,j} + \eta u_{i-1,j}.$$

Đây là phương pháp **explicit**. Muốn tính $u_{i,j+1}$, ta chỉ cần các giá trị ở lớp thời gian cũ j .

4.2 Điều kiện ổn định

Forward-Difference chỉ ổn định khi:

$$\eta = \frac{\alpha^2 k}{h^2} \leq \frac{1}{2}.$$

Nếu $\eta > 0.5$, nghiệm có thể phân kỳ, dao động mạnh hoặc cho giá trị phi vật lý.

Ví dụ nếu $\alpha^2 = 1$ và $h = 0.1$:

$$\frac{k}{0.1^2} \leq \frac{1}{2} \Rightarrow k \leq 0.005.$$

Nếu chọn $k = 0.01$, sơ đồ explicit sẽ không ổn định.

4.3 Độ chính xác

Forward-Difference có sai số xấp xỉ:

$$O(k + h^2).$$

Nghĩa là:

- bậc 1 theo thời gian,
- bậc 2 theo không gian.

Do đó k thường phải chọn khá nhỏ, đặc biệt khi h nhỏ.

4.4 Solver Forward-Difference

```
import numpy as np

def tao_luoi(N_x, N_time, l, t_max):
    x = np.linspace(0.0, l, N_x)
    t = np.linspace(0.0, t_max, N_time)

    h = x[1] - x[0]
    k = t[1] - t[0]

    return x, t, h, k

def tinh_eta(kappa, C, rho, h, k):
    alpha2 = kappa / (C * rho)
    eta = alpha2 * k / h**2
    return eta

def tao_dieu_kien_bien(N_x, N_time, f_init, T_left, T_right, x):
    u = np.zeros((N_x, N_time), dtype=float)

    # Dieu kien dau: u(x,0) = f_init(x)
    u[:, 0] = f_init(x)

    # Dieu kien bien: u(0,t), u(l,t)
    u[0, :] = T_left
    u[-1, :] = T_right
```

```

return u

def solve_forward(u, kappa, C, rho, l, t_max):
    N_x, N_time = u.shape

    x, t, h, k = tao_luoi(N_x, N_time, l, t_max)
    eta = tinh_eta(kappa, C, rho, h, k)

    if eta > 0.5:
        raise ValueError(
            f"Forward method khong on dinh: eta = {eta:.6f} > 0.5. "
            "Hay tang N_time, giam k, hoac giam N_x."
        )

    for j in range(N_time - 1):
        for i in range(1, N_x - 1):
            u[i, j + 1] = (
                (1 - 2*eta) * u[i, j]
                + eta * (u[i + 1, j] + u[i - 1, j])
            )

    return u, x, t, eta

```

5. Backward-Difference method

5.1 Ý tưởng

Backward-Difference dùng sai phân lùi theo thời gian:

$$\frac{\partial u}{\partial t} \approx \frac{u_{i,j} - u_{i,j-1}}{k}.$$

Đạo hàm không gian được lấy tại lớp thời gian mới j :

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}.$$

Thay vào phương trình:

$$\alpha^2 \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} = \frac{u_{i,j} - u_{i,j-1}}{k}.$$

Đặt:

$$\eta = \frac{\alpha^2 k}{h^2}.$$

Ta được:

$$(1 + 2\eta)u_{i,j} - \eta u_{i+1,j} - \eta u_{i-1,j} = u_{i,j-1}.$$

Viết lại theo dạng Gauss-Seidel:

$$u_{i,j} = \frac{\eta}{1 + 2\eta} (u_{i+1,j} + u_{i-1,j}) + \frac{1}{1 + 2\eta} u_{i,j-1}.$$

Đặt:

$$\beta_1 = \frac{\eta}{1 + 2\eta}, \quad \beta_2 = \frac{1}{1 + 2\eta}.$$

Khi đó:

$$u_{i,j} = \beta_1 (u_{i+1,j} + u_{i-1,j}) + \beta_2 u_{i,j-1}.$$

5.2 Dạng ma trận

Backward-Difference là phương pháp **implicit**. Tại bước thời gian j , các giá trị $u_{i-1,j}$, $u_{i,j}$, $u_{i+1,j}$ đều thuộc lớp thời gian mới. Do đó ta phải giải hệ tuyến tính:

$$A\mathbf{u}^{(j)} = \mathbf{u}^{(j-1)}.$$

Ma trận A có dạng tam đường chéo:

$$A = \begin{bmatrix} 1 + 2\eta & -\eta & 0 & \dots & 0 \\ -\eta & 1 + 2\eta & -\eta & \dots & 0 \\ 0 & -\eta & 1 + 2\eta & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & -\eta & 1 + 2\eta \end{bmatrix}.$$

5.3 Tính chất

Ưu điểm:

- ổn định hơn Forward-Difference,
- đối với phương trình nhiệt tuyến tính, thường được xem là ổn định không điều kiện,
- có thể dùng bước thời gian lớn hơn,
- phù hợp với bài toán biến thiên nhanh.

Nhược điểm:

- mỗi bước thời gian phải giải một hệ tuyến tính,
- code phức tạp hơn,
- nếu giải bằng Gauss-Seidel thì cần kiểm tra hội tụ,
- ổn định không có nghĩa là chính xác; nếu k quá lớn, nghiệm bị làm mượt quá mạnh.

5.4 Solver Backward-Difference dùng Gauss-Seidel

```
import numpy as np

def solve_backward_gs(u, kappa, C, rho, l, t_max, N_max=10000, tol=1e-6):
    N_x, N_time = u.shape

    x, t, h, k = tao_luoi(N_x, N_time, l, t_max)
    eta = tinh_eta(kappa, C, rho, h, k)

    beta1 = eta / (1 + 2*eta)
    beta2 = 1 / (1 + 2*eta)

    for j in range(1, N_time):
        # Lay nghiem o buoc thoi gian truoc lam guess ban dau
        u[:, j] = u[:, j - 1]

        # Giu dieu kien bien tai lop thoi gian moi
        u[0, j] = u[0, 0]
        u[-1, j] = u[-1, 0]

        for q in range(1, N_max + 1):
            err_max = 0.0
```

```

for i in range(1, N_x - 1):
    u_old = u[i, j]

    u[i, j] = (
        beta1 * (u[i + 1, j] + u[i - 1, j])
        + beta2 * u[i, j - 1]
    )

    err = abs(u[i, j] - u_old)

    if err > err_max:
        err_max = err

    if err_max < tol:
        break
else:
    print(
        f"Canh bao: buoc thoi gian j = {j} "
        f"khong hoi tu sau {N_max} vong lap."
    )

return u, x, t, eta

```

6. Hàm lưu file kết quả

```

def luu_file_truyen_nhiet(filename, u, x, t):
    N_x, N_time = u.shape

    with open(filename, "w", encoding="utf-8") as file:
        file.write("# Bai toan truyen nhiet 1D\n")
        file.write("# Cot du lieu: j   i   t   x   u\n")
        file.write("#\n")

        for j in range(N_time):
            for i in range(N_x):
                file.write(
                    f"{j:5d} {i:5d} "
                    f"{t[j]:15.8e} {x[i]:15.8e} {u[i, j]:15.8e}\n"
                )
            file.write("\n")

```

7. Ví dụ 1: một thanh ban đầu 100 độ C, hai đầu giữ 0 độ C

Bài toán:

$$\alpha^2 \frac{\partial^2 u}{\partial x^2} = \frac{\partial u}{\partial t}.$$

Điều kiện đầu:

$$u(x, 0) = 100.$$

Điều kiện biên:

$$u(0, t) = 0, \quad u(l, t) = 0.$$

Code Forward-Difference:

```

N_x = 100
N_time = 6000

l = 1.0
t_max = 1800.0

```



```

kappa = 210.0
C = 900.0
rho = 2700.0

T_left = 0.0
T_right = 0.0

x, t, h, k = tao_luoi(N_x, N_time, l, t_max)

def f_init(x):
    return np.full_like(x, 100.0)

u = tao_dieu_kien_bien(N_x, N_time, f_init, T_left, T_right, x)
u_forward, x, t, eta = solve_forward(u, kappa, C, rho, l, t_max)
print("eta =", eta)
luu_file_truyen_nhiet("truyen_nhiet_forward.dat", u_forward, x, t)

```

8. Ví dụ 2: hai thanh tiếp xúc

Hai thanh nối nhau. Thanh trái ban đầu nóng hơn, thanh phải ban đầu lạnh hơn:

$$T_{left} = 100, \quad T_{right} = 50.$$

Hai đầu ngoài giữ ở 0:

$$u(0, t) = 0, \quad u(l, t) = 0.$$

Nếu mỗi thanh dài l_{bar} thì tổng chiều dài:

$$l_{total} = 2l_{bar}.$$

Code Forward-Difference:

```

N_x = 101
N_time = 15000

l_bar = 0.25
l_total = 2.0 * l_bar

t_max = 1800.0

T_left_bar = 100.0
T_right_bar = 50.0

kappa = 210.0
C = 900.0
rho = 2700.0

x, t, h, k = tao_luoi(N_x, N_time, l_total, t_max)

def f_init_2thanh(x):
    return np.where(x <= l_bar, T_left_bar, T_right_bar)

u = tao_dieu_kien_bien(
    N_x=N_x,
    N_time=N_time,
    f_init=f_init_2thanh,
    T_left=0.0,
    T_right=0.0,
    x=x
)

u_forward, x, t, eta = solve_forward(u, kappa, C, rho, l_total, t_max)

```

```
print("eta =", eta)
luu_file_truyen_nhiet("truyen_nhiet_2thanh_forward.dat", u_forward, x, t)
```

Code Backward-Difference:

```
u = tao_dieu_kien_bien(
    N_x=N_x,
    N_time=N_time,
    f_init=f_init_2thanh,
    T_left=0.0,
    T_right=0.0,
    x=x
)

u_backward, x, t, eta = solve_backward_gs(
    u,
    kappa, C, rho,
    l_total, t_max,
    N_max=10000,
    tol=1e-6
)

print("eta =", eta)
luu_file_truyen_nhiet("truyen_nhiet_2thanh_backward.dat", u_backward, x, t)
```

9. Vẽ đồ thị kết quả

9.1 Vẽ nhiệt độ theo vị trí tại vài thời điểm

```
import matplotlib.pyplot as plt

def ve_nhiet_do_theo_x(u, x, t, time_indices):
    plt.figure(figsize=(8, 6))

    for j in time_indices:
        plt.plot(x, u[:, j], label=f"t = {t[j]:.1f} s")

    plt.xlabel("x")
    plt.ylabel("Temperature")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

Ví dụ:

```
ve_nhiet_do_theo_x(
    u_forward,
    x,
    t,
    time_indices=[0, 100, 500, 1000, 3000, 5999]
)
```

9.2 Vẽ heatmap $u(x, t)$

```
def ve_heatmap(u, x, t):
    plt.figure(figsize=(8, 6))

    plt.imshow(
        u.T,
        origin="lower",
        aspect="auto",
        extent=[x[0], x[-1], t[0], t[-1]]
    )

    plt.xlabel("x")
    plt.ylabel("t")
    plt.colorbar(label="Temperature")
    plt.tight_layout()
    plt.show()
```

Dùng:

```
ve_heatmap(u_forward, x, t)
```

10. So sánh Forward-Difference và Backward-Difference

Tiêu chí	Forward-Difference	Backward-Difference
Loại phương pháp	Explicit	Implicit
Công thức thời gian	dùng $u_{i,j+1} - u_{i,j}$	dùng $u_{i,j} - u_{i,j-1}$
Cần giải hệ tuyến tính?	Không	Có
Độ phức tạp code	Dễ hơn	Khó hơn
Điều kiện ổn định	$\eta \leq 1/2$	ổn định hơn, thường không điều kiện
Phù hợp khi	bài toán đơn giản, biến thiên chậm	bài toán biến thiên nhanh, muốn bước thời gian lớn
Nhược điểm chính	phải chọn k rất nhỏ	phải giải hệ tuyến tính mỗi bước

11. Những lưu ý quan trọng trong code

11.1 Thứ tự chỉ số mảng

Nếu viết:

```
u = np.zeros((N_x, N_time))
```

thì:

```
u[i, j]
```

nghĩa là:

i: vị trí không gian
j: thời gian

Không nên comment là:

```
# biến đầu là t, biến thứ 2 là x
```

Mà nên sửa thành:

```
# chỉ số đầu là x, chỉ số thứ hai là t  
# u[i, j] = u(x_i, t_j)
```

11.2 Điều kiện đầu và điều kiện biên có thể đè nhau

Trong bài toán:

$$u(x, 0) = 100,$$

nhưng đồng thời:

$$u(0, t) = u(l, t) = 0.$$

Tại hai góc $u(0, 0)$ và $u(l, 0)$ có thể mâu thuẫn giữa điều kiện đầu và điều kiện biên. Trong mô phỏng truyền nhiệt, thường ưu tiên điều kiện biên:

```
u[:, 0] = 100.0
u[0, :] = 0.0
u[-1, :] = 0.0
```

Như vậy hai đầu thanh luôn bằng 0.

11.3 Forward method cần kiểm tra eta

Nên viết:

```
if eta > 0.5:
    raise ValueError("Không ổn định")
```

Không nên chỉ print rồi return False, vì dễ gây lỗi khi unpack giá trị trả về.

11.4 Backward method cần kiểm tra hội tụ

Với Backward-Difference dùng Gauss-Seidel, cần kiểm tra:

```
if err_max < tol:
    break
```

Nếu sau $N_{\max}$ vòng vẫn không hội tụ, nên báo cảnh báo.

11.5 Ổn định không đồng nghĩa với chính xác

Backward-Difference có thể ổn định với k lớn, nhưng nếu k quá lớn thì nghiệm bị làm mượt quá mạnh. Vì vậy vẫn cần chọn N_{time} đủ lớn để nghiệm có ý nghĩa vật lý.

12. Các dạng toán có thể áp dụng

12.1 Truyền nhiệt trong thanh 1D

$$\frac{\partial u}{\partial t} = \alpha^2 \frac{\partial^2 u}{\partial x^2}.$$

Ví dụ:

- thanh ban đầu nóng đều,
- hai đầu giữ lạnh,
- một đầu nóng một đầu lạnh,
- thanh bị làm nguội theo thời gian.

12.2 Hai thanh tiếp xúc

Mô hình hai thanh nối nhau, ban đầu có nhiệt độ khác nhau. Bài toán mô phỏng quá trình truyền nhiệt qua mặt tiếp xúc.

Ứng dụng:

- cân bằng nhiệt giữa hai vật,
- truyền nhiệt qua interface,
- kiểm tra sự lan truyền nhiệt từ vùng nóng sang vùng lạnh.

12.3 Khuếch tán chất

Nếu $u(x, t)$ là nồng độ chất:

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}.$$

Ứng dụng:

- khuếch tán phân tử,
- khuếch tán ion,
- khuếch tán tạp chất trong vật liệu.

12.4 Khuếch tán xác suất

Nếu $u(x, t) = P(x, t)$ là mật độ xác suất:

$$\frac{\partial P}{\partial t} = D \frac{\partial^2 P}{\partial x^2}.$$

Ứng dụng:

- random walk,
- Brownian motion,
- lan truyền mật độ xác suất.

12.5 Bài toán 2D

$$\frac{\partial u}{\partial t} = \alpha^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right).$$

Ứng dụng:

- truyền nhiệt trên tấm kim loại,
 - khuếch tán trên mặt phẳng,
 - truyền nhiệt trong màng mỏng.
-

13. Chiến lược chọn phương pháp

Dùng **Forward-Difference** khi:

- bài toán đơn giản,
- muốn code nhanh,
- N_x không quá lớn,
- có thể chọn N_{time} đủ lớn để thỏa $\eta \leq 1/2$.

Dùng **Backward-Difference** khi:

- bài toán cần bước thời gian lớn,
 - nghiệm thay đổi nhanh,
 - Forward-Difference đòi hỏi N_{time} quá lớn,
 - chấp nhận phải giải hệ tuyến tính ở mỗi bước thời gian.
-

14. Công thức cần nhớ

Hệ số khuếch tán nhiệt:

$$\alpha^2 = \frac{K}{C\rho}.$$

Bước lưới:

$$h = \frac{l}{N_x - 1}, \quad k = \frac{t_{\max}}{N_t - 1}.$$

Tham số ổn định:

$$\eta = \frac{\alpha^2 k}{h^2}.$$

Forward-Difference:

$$u_{i,j+1} = (1 - 2\eta)u_{i,j} + \eta u_{i+1,j} + \eta u_{i-1,j}.$$

Điều kiện ổn định Forward-Difference:

$$\eta \leq \frac{1}{2}.$$

Backward-Difference:

$$(1 + 2\eta)u_{i,j} - \eta u_{i+1,j} - \eta u_{i-1,j} = u_{i,j-1}.$$

Dạng Gauss-Seidel:

$$u_{i,j} = \frac{\eta}{1 + 2\eta} (u_{i+1,j} + u_{i-1,j}) + \frac{1}{1 + 2\eta} u_{i,j-1}.$$

15. Kết luận

Phương trình truyền nhiệt là ví dụ điển hình của **parabolic equation**. Với bài toán 1D, ta có thể giải bằng sai phân hữu hạn theo hai hướng chính:

- **Forward-Difference method**: explicit, đơn giản, dễ code, nhưng phải thỏa điều kiện ổn định $\eta \leq 1/2$.
- **Backward-Difference method**: implicit, ổn định hơn, phù hợp với bài toán thay đổi nhanh, nhưng phải giải hệ tuyến tính tại mỗi bước thời gian.

Trong thực hành code cần chú ý:

- thứ tự chỉ số $u[i, j]$,
- cách gán điều kiện đầu và điều kiện biên,
- kiểm tra η ,
- kiểm tra hội tụ nếu dùng Backward-Difference,
- ổn định không đồng nghĩa với chính xác.

Tài liệu nguồn

- Bài giảng PDF: *Computational physics - Partial Differential Equations - Numerical solutions to parabolic equations*.
- Code thực hành: Forward-Difference và Backward-Difference solver cho bài toán truyền nhiệt 1D và hai thanh tiếp xúc.